

Actividad de Clase: Funciones Definidas por el Usuario en PostgreSQL

Eres analista de datos en una empresa de retail que gestiona información de ventas en una base de datos PostgreSQL. Para mejorar la eficiencia de los reportes, tu equipo necesita encapsular cálculos frecuentes en funciones definidas por el usuario (UDFs) en SQL. Tu tarea será crear y probar funciones en PostgreSQL que automaticen consultas como el cálculo de impuestos, descuentos e ingresos por cliente.

Objetivo y Preparación de la Actividad

Identificar componentes

Reconocer los elementos de una función definida por el usuario en SQL y sus casos de uso prácticos

Implementar funciones

Crear funciones definidas por el usuario (SQL Functions) en PostgreSQL para automatizar cálculos

Aplicar a negocio

Usar las funciones en tablas de ventas para generar indicadores de negocio relevantes

Materiales y preparación previa (checklist):

- Cliente PostgreSQL (pgAdmin, DBeaver u otro)
- Base de datos con la tabla ventas
- Acceso a servidor PostgreSQL
- Permisos para crear funciones (CREATE FUNCTION)

```
CREATE TABLE ventas (  
  id SERIAL PRIMARY KEY,  
  cliente TEXT,  
  producto TEXT,  
  cantidad INT,  
  precio_unitario NUMERIC,  
  impuesto NUMERIC,  
  fecha DATE  
);
```

```
INSERT INTO ventas (cliente, producto, cantidad, precio_unitario, impuesto, fecha) VALUES  
(  
  'Juan Pérez', 'Laptop', 2, 1200, 0.19, '2023-06-15'),  
(  
  'Ana Torres', 'Teléfono', 1, 800, 0.19, '2023-06-20'),  
(  
  'Juan Pérez', 'Tablet', 3, 500, 0.10, '2023-07-01');
```

Instrucciones Paso a Paso

01

Revisión teórica

- Identifica los componentes de una función (nombre, parámetros, tipo de retorno, cuerpo, lenguaje, volatilidad). ✓
- Discute con un compañero al menos dos casos de uso donde una empresa prefiera encapsular lógica en funciones SQL.

03

Función 2: total de venta

- Implementa `total_venta(cantidad INT, precio NUMERIC)` que devuelva el valor total de una venta. ✓
- Prueba: `SELECT total_venta(5,120);`

05

Función 4: rango de fechas (RETURNS TABLE)

- Implementa `ventas_rango.fecha_inicio DATE, fecha_fin DATE)` que retorne cliente y total de ventas en el período. ✓
- Prueba: `SELECT * FROM ventas_rango('2023-06-01','2023-07-31');`

02

Función 1: cálculo simple

- Crea la función `calcular_impuesto(precio NUMERIC, impuesto NUMERIC)` que devuelva el precio final. ✓
- Prueba: `SELECT calcular_impuesto(100,0.19);`

04

Función 3: ingresos de un cliente

- Crea `ingresos_cliente(nombre_cliente TEXT)` que retorne el total de ingresos generados por un cliente. ✓
- Prueba: `SELECT ingresos_cliente('Juan Pérez');`

06

Entrega

Sube al Moodle un archivo .sql con las funciones implementadas y capturas de pantalla de su ejecución.

Criterios de Éxito y Errores Comunes

Criterios de éxito:

- Todas las funciones compilan y entregan resultados correctos
- Se incluyen comentarios (-- explicación de la función)
- Se muestran pruebas de ejecución con resultados esperados

Errores comunes:

- No declarar correctamente el tipo de retorno
- Omitir parámetros de entrada o usarlos mal
- Confundir RETURNS SCALAR con RETURNS TABLE

Rúbrica de Evaluación

Criterio	Excelente (25%)	Competente (20%)	Básico (15%)	Ponderación
1. Comprensión del concepto clave	Explica claramente componentes de una función y casos de uso, con ejemplos de negocio.	Explica parcialmente componentes de una función.	Menciona solo algunos conceptos sin conexión práctica.	25%
2. Aplicación técnica	Todas las funciones funcionan correctamente y devuelven resultados correctos.	Funciones se ejecutan con errores menores o limitaciones.	Solo una parte de las funciones funciona o resultados incompletos.	30%
3. Calidad del entregable	Entrega archivo .sql documentado y pruebas de ejecución completas.	Entrega archivo con funciones pero sin comentarios claros.	Entrega incompleta o sin pruebas.	20%
4. Justificación/interpretación	Analiza cuándo conviene usar funciones en SQL con ejemplos claros.	Menciona al menos un caso de uso.	Explica de manera vaga sin conexión empresarial.	25%

Total: 100%